

Design and Analysis of Algorithms: CS 218

Sujoy Bhore

Indian Institute of Technology Bombay

Instructor



- Sujoy Bhore
- `sujoy@cse.iitb.ac.in`
- Office: Room no. 318,
New CSE/CC Building
- Office hours: tbd.

Instructor



- Sujoy Bhore
- `sujoy@cse.iitb.ac.in`
- Office: Room no. 318,
New CSE/CC Building
- Office hours: tbd.

Research Interests: Algorithms, Discrete & Computational Geometry, Combinatorial Optimization, Theoretical Machine Learning.

Teaching Assistants (TAs)

PG TAs

- Devdan Dey (23D0365); Email - sandman@cse.iitb.ac.in.
- Himanshi Singh (23D0363); Email - himanshi@cse.iitb.ac.in.
- Prateek Chanda (22D0362); Email - prateekch@cse.iitb.ac.in.
- Pranav Kandwal (24M0771); Email - pranavk@cse.iitb.ac.in.
- Tejas Sanjay Shinde (24M0827); Email - tejasshinde@cse.iitb.ac.in.
- Vadapalli Hemanth Venkata Sai (25M2118); hemanthvs@cse.iitb.ac.in.
- Mayank Jha (25M0762); Email - mayankiitbcse@cse.iitb.ac.in.
- Sakshi Shivkumar Pandey (24M2115); Email - sakshipandey@cse.iitb.ac.in.

Teaching Assistants (TAs)

UG TAs

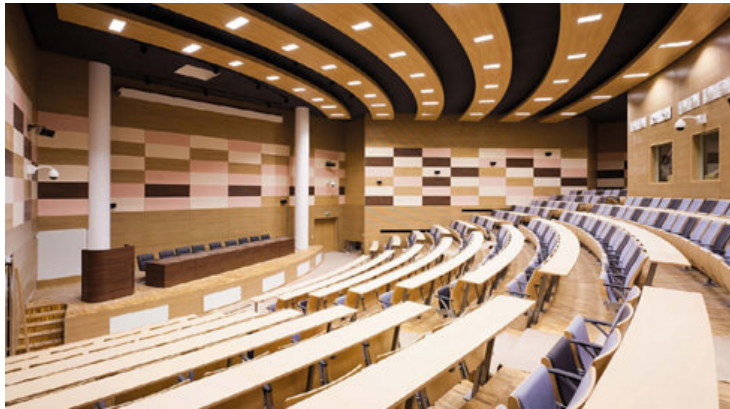
- Ved Danait (22b1818); Email - veddanait@cse.iitb.ac.in.
- Mayank Motwani (22b1052); Email - mayankmotwani@cse.iitb.ac.in.
- Deeksha Dhiwakar (22b0988); Email - deeksha@cse.iitb.ac.in.
- Mrinal Garg (22b0930); Email - mrinalgarg@cse.iitb.ac.in.
- Anilesh Bansal (22b0928); Email - anileshbansal@cse.iitb.ac.in.

Schedule

- Monday, 11:35:00 - 12:30:00. LA 001.
- Tuesday, 08:30:00-09:25:00. LA 001.
- Thursday, 09:30:00-10:25:00. LA 001.

Schedule

- Monday, 11:35:00 - 12:30:00. LA 001.
- Tuesday, 08:30:00-09:25:00. LA 001.
- Thursday, 09:30:00-10:25:00. LA 001.



★ participation in offline classes is mandatory.

Blended Learning ...

- Course content mostly via offline classes, or literature reading, and reading materials.
- Self-learning units and quizzes will be released regularly.
- Once in every two weeks, Tutorial session to answer questions about previous units or discuss further details.

Blended Learning ...

- Course content mostly via offline classes, or literature reading, and reading materials.
 - Self-learning units and quizzes will be released regularly.
 - Once in every two weeks, Tutorial session to answer questions about previous units or discuss further details.
-
- Two exercise sheets to solve and hand in
 - Four surprise quizzes.
 - Mid-sem and End-sem exams.
- ★ participation in offline classes is mandatory.

Blended Learning ...

- Course content mostly via offline classes, or literature reading, and reading materials.
- Self-learning units and quizzes will be released regularly.
- Once in every two weeks, Tutorial session to answer questions about previous units or discuss further details.

Quizzes: tbd.

Tentative:

sheet	date	time
1	31.01.	9:00 PM
2	15.03.	9:00 PM

- Two exercise sheets to solve and hand in
 - Four surprise quizzes.
 - Mid-sem and End-sem exams.
- ★ participation in offline classes is mandatory.

Organization

- see the link -
<https://sites.google.com/view/cs218designandanalysisofalgori/home>
- will use moodle for the announcement.
- register for the CS 218 course.

Organization

- see the link -
<https://sites.google.com/view/cs218designandanalysisofalgori/home>
- will use moodle for the announcement.
- register for the CS 218 course.
- Up-to-date course information
- Links to meetings (in case any online session)
- Lecture slides + notes
- Discussion forum
- Exercise sheets
- Additional literature and material
- Registration for exercise group.

Organization

- see the link -
<https://sites.google.com/view/cs218designandanalysisofalgori/home>
- will use moodle for the announcement.
- register for the CS 218 course.
- Up-to-date course information
- Links to meetings (in case any online session)
- Lecture slides + notes
- Discussion forum
- Exercise sheets
- Additional literature and material
- Registration for exercise group.
- Crib sessions will be conducted in a blended mode. Please take clear photographs of your written solutions during these sessions, and submit your questions through the Google Form. We will review all submissions and address each concern.

Goals and Requirements

Objectives: At the end of the course you will be able to...

- explain concepts, structures, and problem definitions.
- understand the discussed algorithms, explain and analyze them.
- select and adapt appropriate algorithms and data structures.
- analyze new algorithmic problems and develop efficient solutions.

Goals and Requirements

Objectives: At the end of the course you will be able to...

- explain concepts, structures, and problem definitions.
- understand the discussed algorithms, explain and analyze them.
- select and adapt appropriate algorithms and data structures.
- analyze new algorithmic problems and develop efficient solutions.

Prior Knowledge:

- Data structures and algorithms (CS 213 + 293).
- Discrete Structures (CS 105).

Goals and Requirements

Objectives: At the end of the course you will be able to...

- explain concepts, structures, and problem definitions.
- understand the discussed algorithms, explain and analyze them.
- select and adapt appropriate algorithms and data structures.
- analyze new algorithmic problems and develop efficient solutions.

Prior Knowledge:

- Data structures and algorithms (CS 213 + 293).
- Discrete Structures (CS 105).

Course Time Breakdown:

- offline classes, learning materials, and exercise sessions
- self-study of lecture material and working on exercises
- exam and exam preparation

Final Grading

- Four Quizzes - 20%
- Two Assignments - 20%
- Mid Sem exam - 20%
- End Sem exam - 35%
- Attendance - 5%

Final Grading

- Four Quizzes - 20%
- Two Assignments - 20%
- Mid Sem exam - 20%
- End Sem exam - 35%
- Attendance - 5%

Requirements for positive grade

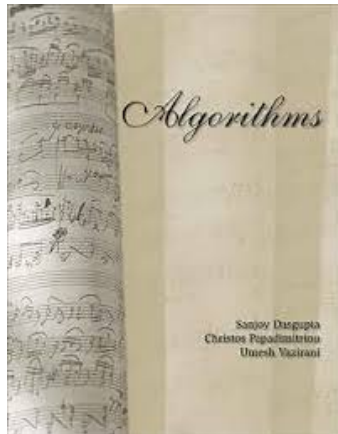
Successful participation in exercise sessions, in particular:

- hand in the assignments.
- do the programming assignments (if given).
- do not miss classes unexcused.
- pass in the final exam.

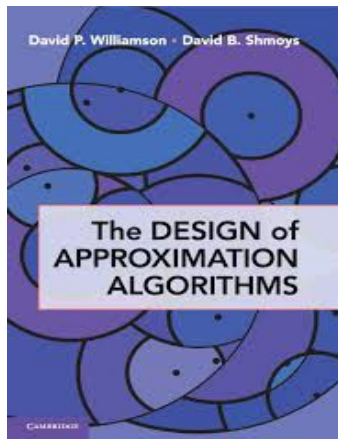
Class Materials



Algorithms Design.
John Kleinberg, Eva Tardos.

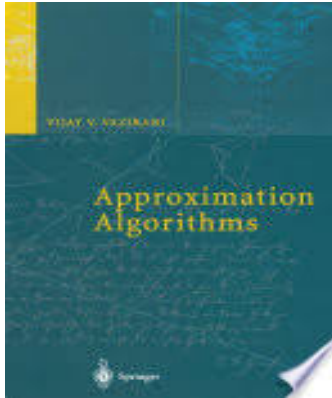


Algorithms.
Sanjoy Dasgupta, Christos Papadimitriou, and
Umesh Vazirani.

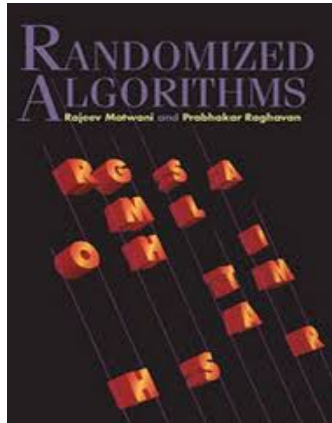


The Design of Approximation Algorithms,
David B. Shmoys and David P. Williamson.

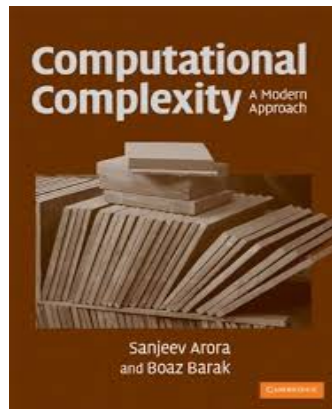
Class Materials



Approximation Algorithms,
Vijay Vazirani.



Randomized algorithms,
Rajeev Motwani and Prabhakar Raghavan.



Computational Complexity: A Modern
Approach,
Sanjeev Arora and Boaz Barak.

What is an Algorithm?

A finite sequence of “well-defined” instructions to solve a given computational problem.

What is an Algorithm?

A finite sequence of “well-defined” instructions to solve a given computational problem.



What is an Algorithm?

A finite sequence of “well-defined” instructions to solve a given computational problem.



- Primary objective: Design of efficient algorithms; Analyze the Algorithms.
- Additional objective: Implement those efficient algorithms

Journey of Algorithms

- Around 825 AD, Persian polymath Muhammad ibn Musa al-Khwarizmi wrote *Kitab al-hisab al-hindi* (“Book of Indian computation”) and *Kitab al-jam’ wa’l-tafriq al-hisab al-hindi* (“Addition and subtraction in Indian arithmetic”).



- Medieval period: algorism = arithmetic using Hindu–Arabic numerals.
- Later: spelling shifted to algorithm (influenced by Greek arithmos = number).
- Today: a finite, well-defined procedure for solving a problem.

Art of Optimization!

Stock markets



Drug design



Self-Driving Cars



Chip Design

Art of Optimization!

Stock markets



Drug design



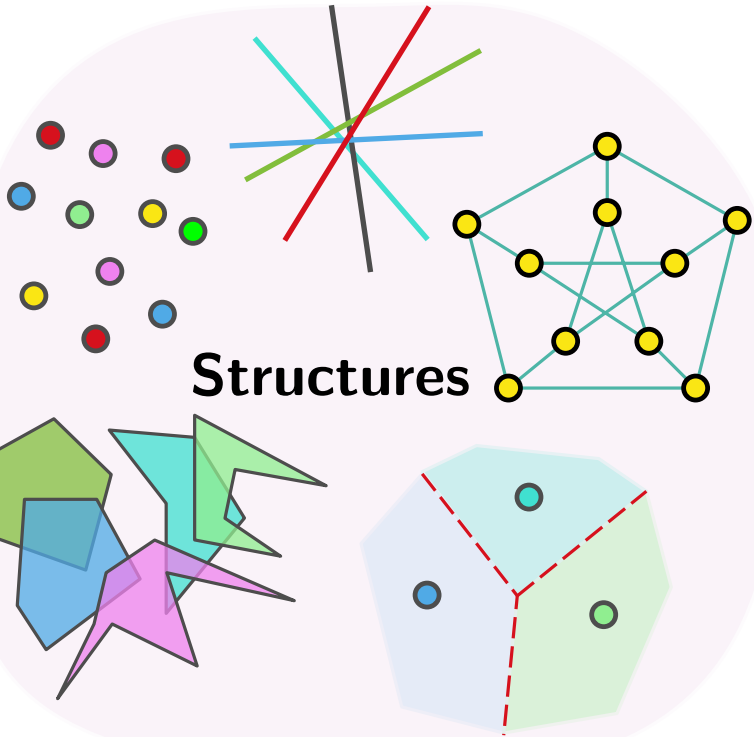
Self-Driving Cars



Chip Design

Computing ...

Structures



Combinations

$$f_{k-1} = \sum_{i=k}^d (-1)^{d-i} \binom{i}{k} f_{i-1}.$$

$$N(\mathcal{A}) = \sum_{x \in L(\mathcal{A})} (-1)^{r(x)} \mu(0, x)$$

$$\#(tP \cap \mathbb{Z}^n) = \sum_{i=0}^d a_i t^i,$$

$$\chi_T(\phi+2) = (\phi+2) \phi^{3V(T)-10} (\chi_T(\phi+1))^2$$

$$\chi(KG_{n,k}) = n - 2k + 2$$

$$\frac{2(2n)13^n}{n!(n+2)!}$$

$$\frac{\lambda_1 \lambda_2 \dots \lambda_{n-1}}{n}$$

$$\frac{\alpha(G)}{n} \leq \frac{\lambda_{\min}}{d - \lambda_{\min}}$$

$$(1 - \lambda_2)/2 \leq h(G) \leq \sqrt{2(1 - \lambda_2)},$$

$$\Theta(C_5) = \sqrt{5}.$$

$$\mu_c := \lim_{n \rightarrow \infty} c_n^{1/n} = \sqrt{2 + \sqrt{2}}.$$

$$\ell_n \sim 2\sqrt{n}.$$

$$\lim_{n \rightarrow \infty} \text{Prob} \left(\frac{\text{is}(\sigma) - 2\sqrt{n}}{n^{1/6}} \leq t \right) = F(t).$$

$$b \geq v.$$

$$\text{per}(A) \geq \frac{n!}{n^n}$$

$$W(C^\perp; x, y) = \frac{1}{|C|} W(C; y - x, y + x).$$

$$|C| \leq \sum_{i=0}^d \binom{n}{i},$$

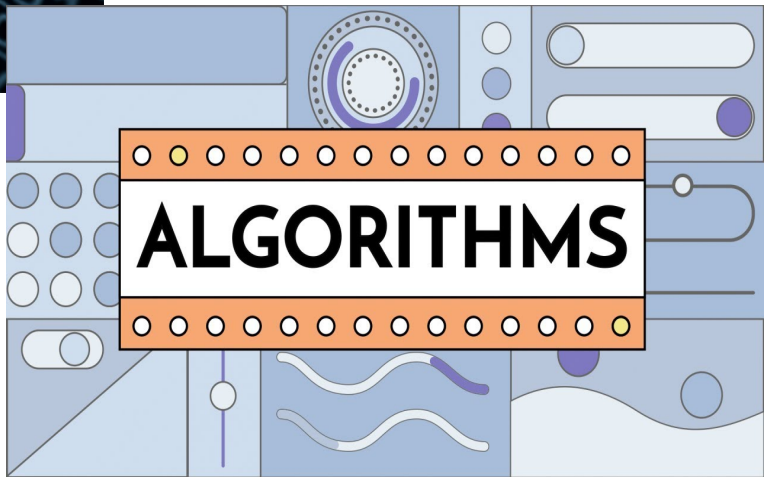
$$(2/e)(1+o(1))k2^{k/2} \leq r(k+1, k+1) \leq k^{-C \log k / \log \log k} \binom{2k}{k}.$$

$$\aleph_{\omega}^{\aleph_0} < \max\{\aleph_{\omega_1}, (2^{\aleph_0})^+\},$$

$$\max(|A+A|, |A \times A|) \geq \frac{1}{2} |A|^{4/3} (\log |A|)^{-1/3}.$$

$$\aleph_0 \leq^* p \iff \aleph_0 \leq 2^p$$

$$Q_n = 2n(\ln n + \gamma - 2) + 2 \ln n + 2\gamma + 1 + O\left(\frac{1}{n}\right)$$

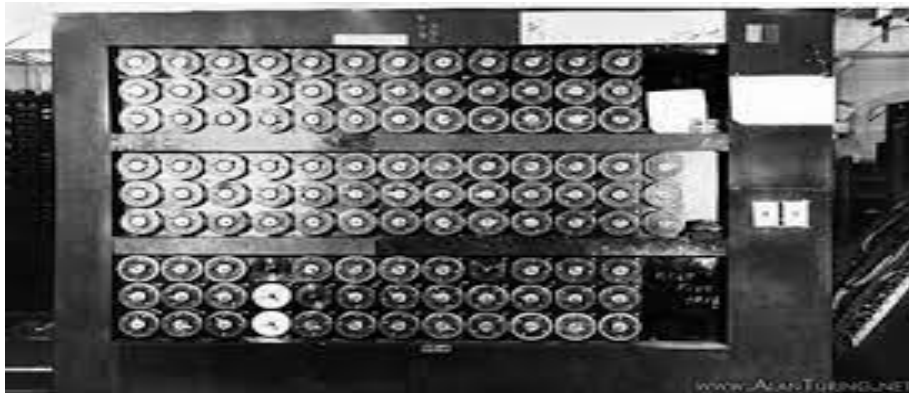


Computing ...

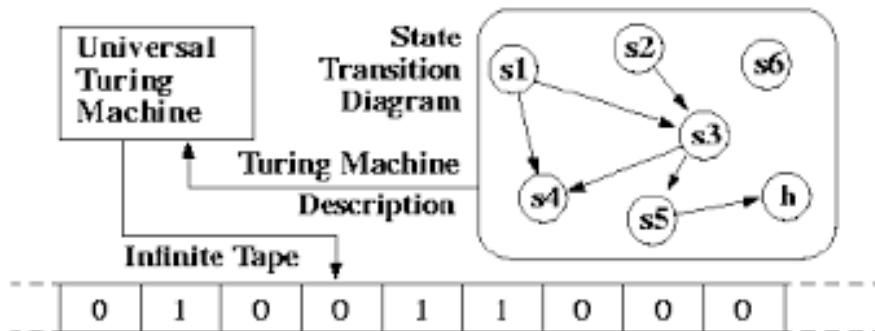
Computer science is no more about computers than astronomy is about telescopes, biology is about microscopes or chemistry is about beakers and test tubes. Science is not about tools, it is about how we use them and what we find out when we do... – Dijkstra/ folklore ... ?

Computing ...

Computer science is no more about computers than astronomy is about telescopes, biology is about microscopes or chemistry is about beakers and test tubes. Science is not about tools, it is about how we use them and what we find out when we do... – Dijkstra/ folklore ... ?

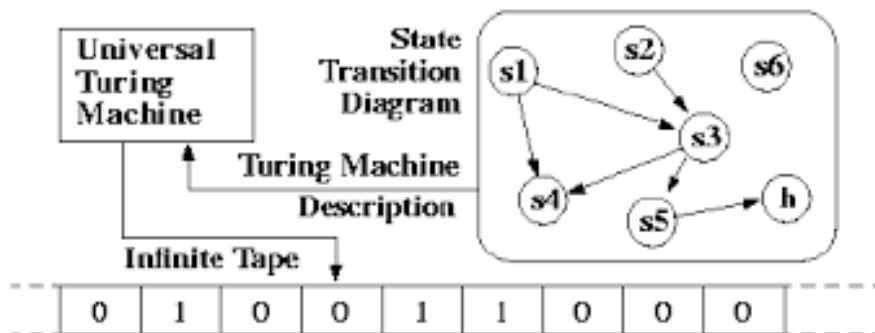
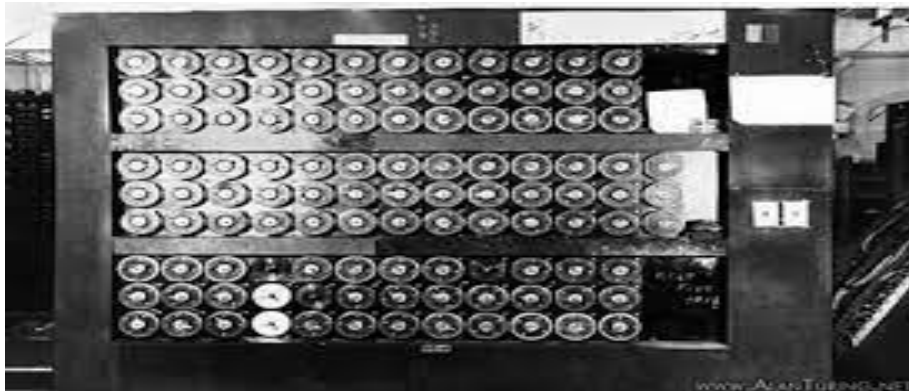


Turing era ...

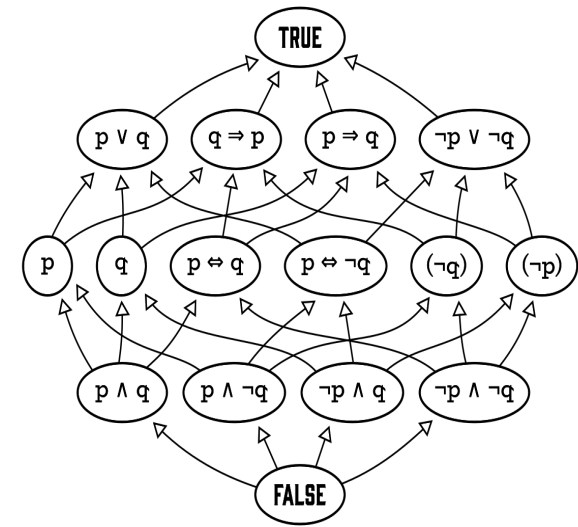


Computing ...

Computer science is no more about computers than astronomy is about telescopes, biology is about microscopes or chemistry is about beakers and test tubes. Science is not about tools, it is about how we use them and what we find out when we do... – Dijkstra/ folklore ... ?



Turing era ...



we are building upon the 2500 yrs. of knowledge ...

Algorithms in Modern Age!

Important for All Branches of Computer Science.

Networks
graph algos.

Robotics & Graphics
geometric algos.

AI & ML
continuous
optimization

Cryptography
number theoretic
algos.

Databases
fast
data-structures

Algorithms in Modern Age!

Important for All Branches of Computer Science.

Networks
graph algos.

Robotics & Graphics
geometric algos.

AI & ML
continuous
optimization

Cryptography
number theoretic
algos.

Databases
fast
data-structures

“Lens” beyond Computer Science!

Physics
quantum
computation

Economics
understanding
“equilibria”

Biology
evolution as a
search algorithm

Art of Designing Algorithms

- What is a “good” algorithm?

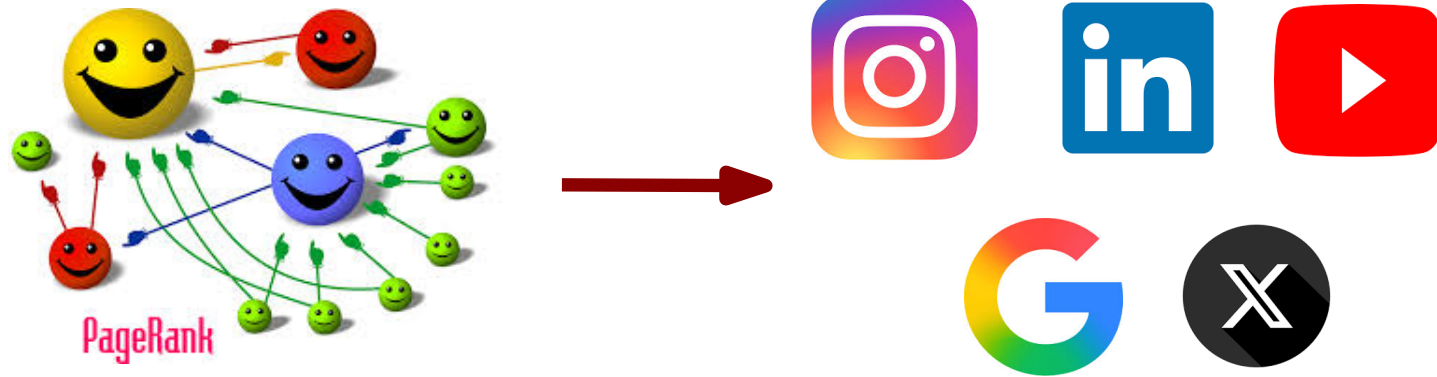
Art of Designing Algorithms

- What is a “good” algorithm?

- **Correctness:** The algorithm must be correct, including all corner cases ...
- **Efficiency:** Economical in use of time, space and other resources
- **Well-documented and with sufficient details:**
- **Maintainable:**
 - Easy to understand, clear & concise (not tricky)
 - Easy to modify (if necessary)
 - Easily understood at different levels
 - Not System (computer) dependent
 - Usable as modules by others

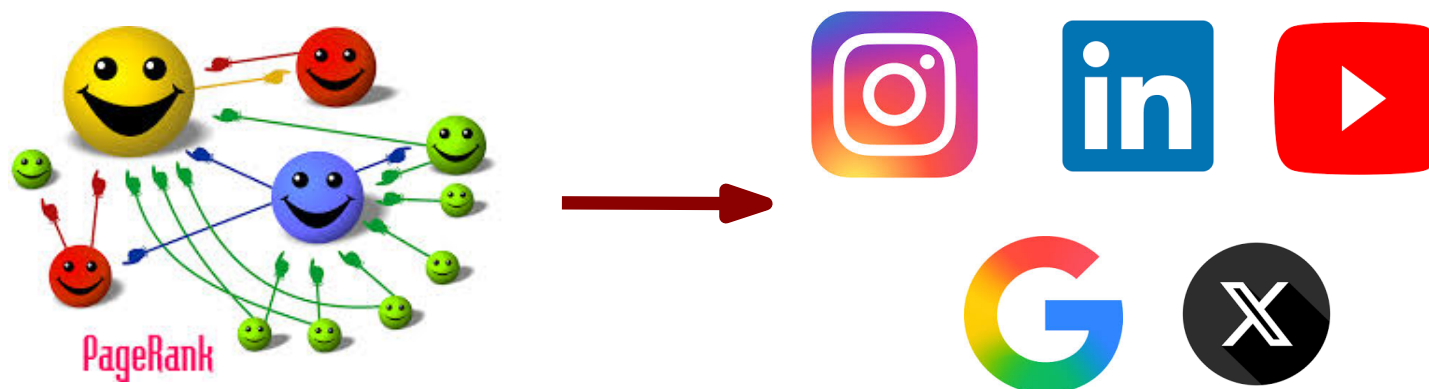
Art of Designing Algorithms

- What is a “good” algorithm?



Art of Designing Algorithms

- What is a “good” algorithm?

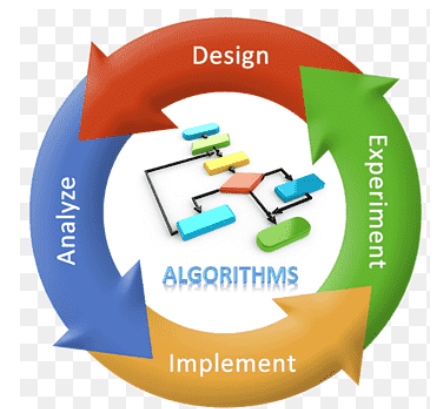


- Algorithm design is an art that demands a lot of creativity, intuition, and perseverance
- Efficient algorithms are important for theoretical as well as practical purposes.
- There are many practically relevant problems for which there does not exist any efficient algorithm till date (How to deal with them?)

How to design Efficient Algorithms?

Facts from the world of algorithms:

- No ONE formula for designing efficient algorithms. (Good thing: There are some standard paradigms/techniques (some of) which we will learn in this course...)
- Every new problem demands a fresh approach
- Designing an efficient algorithm or data structure requires ...
 - Ability to make key observations
 - Ability to ask right kind of questions
 - A positive attitude and ...
 - A lot of perseverance



Simplicity vs. Efficiency

Two goals in designing an algorithm:

Simplicity

and/or

Efficiency

Easy to code
Easy to understand
Easy to debug

Faster
Use less space

Simplicity vs. Efficiency

Two goals in designing an algorithm:

Simplicity

and/or

Efficiency

Easy to code

Easy to understand

Easy to debug

Faster

Use less space

Usually, the two goals contradict each other!

- A naive simple algorithm is (usually) slower
- A fast algorithm tends to be complicated

Simplicity vs. Efficiency

Two goals in designing an algorithm:

Simplicity

and/or

Efficiency

Easy to code

Easy to understand

Easy to debug

Faster

Use less space

Usually, the two goals contradict each other!

- A naive simple algorithm is (usually) slower
- A fast algorithm tends to be complicated

Depends on two questions:

- How often does the problem occur? Only a few times? or very often?
- How big is the problem? Small, medium, or big?

Example: Fibonacci Number $f(n)$

- $f(0) = 0$
 - $f(1) = 1$
 - $f(n) = f(n - 1) + f(n - 2)$ for $n \geq 1$
-
- Used by Pingala to count the number of patterns of short and long vowels.
 - Here we are repeating the same operation n times.
 - Can we compute $F(n)$ in $O(\log n)$ arithmetic operations?

Will revisit again!

Example: Integer Multiplication

- **Addition:**

- Adding two n bit numbers
(School method $O(n)$ time.)
- $O(n)$ time is necessary to write the output.

- **Multiplication:**

- Multiplying two n bit numbers.
(School method $O(n^2)$ time)
- is $O(n^2)$ time is necessary?

Will revisit again!

Analysis of an Algorithm

- There are two issues:
 - The running time of the algorithm: The amount of storage/space used by the algorithm
 - We will focus primarily on running time. Storage/space can be analyzed similarly. Reason: most storage requirements in classic algorithms is $O(n)$
 - What is the trivial way to analyze running time of an algorithm?
- **Simulation:** Run the algorithm many times and measure the running time.
- **Mathematical analysis!**

Analysis of an Algorithm

- There are two issues:
 - The running time of the algorithm: The amount of storage/space used by the algorithm
 - We will focus primarily on running time. Storage/space can be analyzed similarly. Reason: most storage requirements in classic algorithms is $O(n)$
 - What is the trivial way to analyze running time of an algorithm?
- **Simulation:** Run the algorithm many times and measure the running time. Machine dependent - Bad!
- **Mathematical analysis!** Robust!

Analysis of an Algorithm

- There are two issues:
 - The running time of the algorithm: The amount of storage/space used by the algorithm
 - We will focus primarily on running time. Storage/space can be analyzed similarly. Reason: most storage requirements in classic algorithms is $O(n)$
 - What is the trivial way to analyze running time of an algorithm?
- **Simulation:** Run the algorithm many times and measure the running time. Machine dependent - Bad!
- **Mathematical analysis!** Robust!

Model of computation - Word-RAM model... many parameters at play - but don't affect heavily most of the algos.!

Algorithms and Complexity!

Designing fast
algorithms

- Divide-and-Conquer
- Graph Algorithms
- Greedy Algorithms
- Dynamic Programming
- Network Flow
- Linear Programming

Algorithms and Complexity!

Designing fast
algorithms

- Divide-and-Conquer
- Graph Algorithms
- Greedy Algorithms
- Dynamic Programming
- Network Flow
- Linear Programming

Proving that a fast
algorithm (prob.)
doesn't exist

- NP-Completeness and beyond!

Algorithms and Complexity!

Designing fast algorithms

- Divide-and-Conquer
- Graph Algorithms
- Greedy Algorithms
- Dynamic Programming
- Network Flow
- Linear Programming

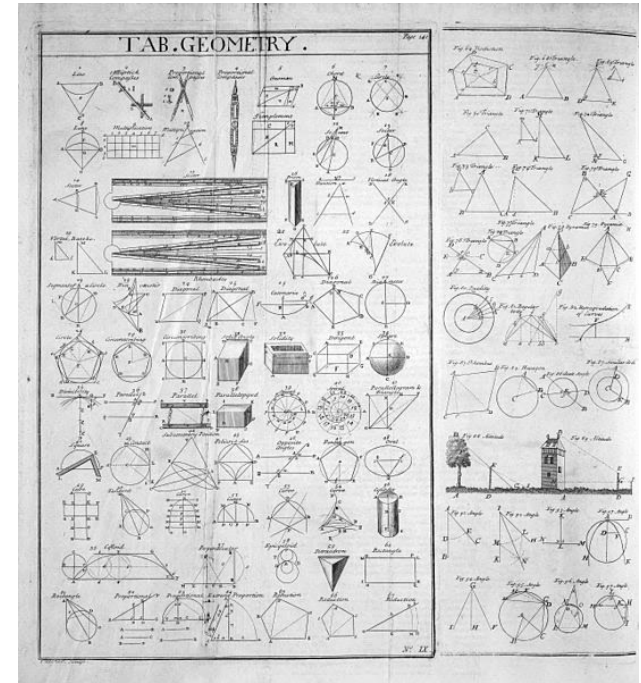
Proving that a fast algorithm (prob.) doesn't exist

- NP-Completeness and beyond!

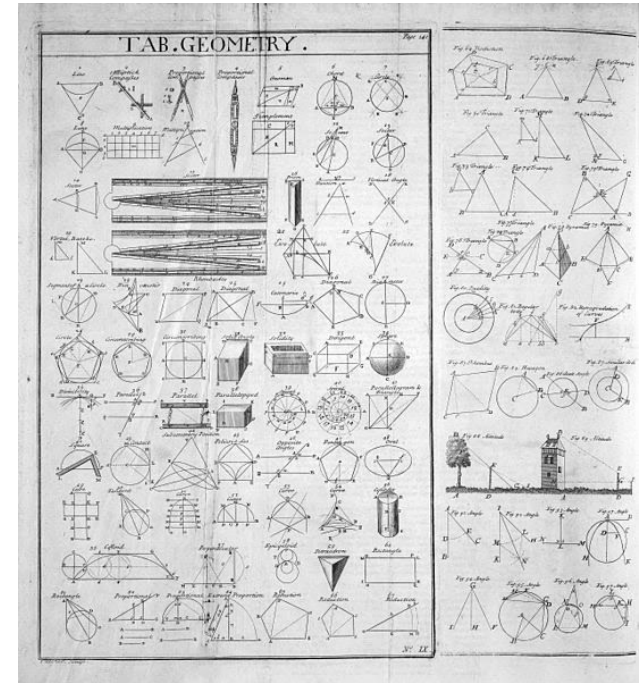
Coping with hardness

- Approximation
- Randomization
- Parameterization

Let's begin the learning
together ...



Let's begin the learning
together ...



Thank you!